

1 주차 진행 공유

시험 벼락치기 스케줄러

시험기간 대학생들을 위한 수면 · 카페인 스케줄 최적화 도구

N110_ 안정연

왜 이 프로젝트를 시작했나



상황

시험기간이 되면 대학생 대부분은 벼락치기를 하고,
수면 · 카페인 섭취를 순전히 감과 스트레스에 의존해
조절한다.

나의 경험

스트레스로 매운 음식 · 결식 반복

→ 카페인 과다 섭취 → 수면 부족 → 위염

정작 시험 당일엔 컨디션 저하로 죽만 먹는 경험

핵심 문제

- 1 언제 자고, 언제 일어나고, 카페인을 얼마나 마셔야 할지
과학적 근거 없이 직관에만 의존
- 2 그 결과 수면 부족 누적 · 위장 장애 · 시험 당일
컨디션 저하라는 역효과 발생
- 3 시험이 하루가 아니라 일주일 내내 겹치는데,
"오늘 밤"만 생각하고 시험 기간 전체 배분은 못 함
- 4 일부 학생의 특수 상황이 아니라,
시험 있는 대학생이라면 학기마다 겪는 보편적 문제

시험 여러 개가 겹치는 "시험 주간" 전체를 놓고 컨디션을 배분해주는 기능

프로젝트 로드맵 (4 주)

1 주차

- 아이디어 검증 및 문제 정의
- 기획서 작성 (v1 → v2)
- 디자인 문서 · 스킬 설정
- 프로토타입 제작 → React 이식

진행 중

2 주차

- 줄음 · 각성 리듬 계산 공식 구현
- 카페인 효과 계산 방식 만들기
- 카페인 안전량 체크 기능
- 시험 여러 개 한번에 고려하기

다음 순서

3 주차

- 화면에 계산 기능 실제 연결
- 그래프로 결과 보여주기
- 서버 · DB 데이터 연동

예정

4 주차

- 추가 기능 (여유 시)
- 전체 테스트 · 버그 수정
- 발표 자료 · 데모 준비

예정

설계 과정: 아이디어 검증 → 전공 지식 (수면과학 논문) 기반 설계 → 시나리오 · 기획서 완성 → 4 주 개발

이번 주 진행 상황 (7/6 ~ 7/9)

7/6 (월)

- React 개발 환경 설정
- 프로토타입 인터페이스 레이아웃 디자인 착수

7/7 (화)

- exam-cram-scheduler 와이어프레임 프로토타입 제작
- 기획서 초안 작성

7/8 (수)

- 기획서 (v2) 완성
- HTML/CSS 프로토타입 5 개 화면 완성 (prototype_v1)
- GitHub Pages 자동 배포 워크플로 연결

7/9 (목)

- 서비스 기술 지도 · CLAUDE.md · 디자인 문서 정리
- 프로토타입 5 개 화면을 React 앱으로 이식

서비스 기술 지도



FE — React

입력 폼 · 캘린더 UI, 각성도 그래프 · 타임라인 시각화,
계산 결과는 브라우저 localStorage에 저장



BE — Express

입력값을 Two-Process + UMP 모델로 계산하는 API,
음료별 카페인 함량 등 참고 데이터 제공 API



DB — Supabase (Postgres)

카페인 함량표 · 반감기 · 안전 섭취 한도 등
참고용 정적 데이터를 저장

핵심 기능 하나 — 계산하기 요청 흐름



디자인 — 컬러 & 타이포그래피

컬러 팔레트

	brand	#9A5B28	주요 버튼 · 강조
	brand-strong	#7A4720	진한 강조 텍스트
	brand-soft	#F1E0C9	연한 배경 · 아이콘 배경
	ink-900	#201409	기본 텍스트
	ink-600	#786A5C	보조 텍스트
	surface-muted	#F6F1E9	카드 배경
	tag-sleep	#43290F	수면 아이콘 / 범례
	tag-study	#B98A55	공부 아이콘 / 범례
	tag-caffeine	#D99A3D	카페인 아이콘 / 범례

타이포그래피 (Pretendard / Noto Sans KR)

display-num	28px / 800	"D-2 · 생화학 "
headline	19px / 800	" 상황을 알려주세요 "
section-head	15px / 700	" 시험 일정 "
row-title / row-sub	14.5px · 12.5px	" 세포생물학 " / " 월 09:00 "
btn	14.5px / 700	" 계산하기 "
여백 · 모서리 규칙 카드 / 차트 22px · 입력박스 16px · 필드 10px 라운드, 화면 좌우 24px 여백		

디자인 변화 — Before / After

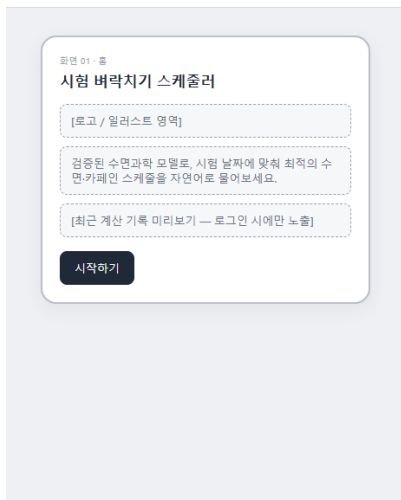
Before · 초기 와이어프레임 (docs/screens/)

시험 벡락치기 스케줄러 · 와이어프레임

화면 목록

화면 흐름 1. 홈 2. 정보 입력 3. 처리 중 4. 결과

5. 스케줄 조정



After · 현재 client/ 프로토타입 (실제 렌더링)

● 시험 벡락치기 스케줄러

가장 가까운 시험

D-2 · 생화학

금요일 10:00 시작 · 지금부터 계산하면 취침·기상·카페인 스케줄을 바로 알려드려요.

최근 계산 기록

이 브라우저에 저장됨



이렇게 계산해요



새 스케줄 만들기

무엇이 바뀌었나 : 회색 점선 placeholder → 브랜드 컬러 (#9A5B28) 적용 , 실제 아이콘 · 카드 · 둥근 모서리 컴포넌트로 구현

React 컴포넌트 & 폴더 구성

구현된 컴포넌트 — client/src/components

```
client/src/components/  
├ Card/           (Card.tsx + .module.css)  
├ Row/  
├ Segmented/  
├ Switch/  
├ Slider/  
├ Button/  
├ Field/  
├ WarningBanner/  
├ BottomSheet/  
├ icons.tsx  
└ index.ts       (모아서 export)
```

9 개 컴포넌트 모두 *PascalCase.tsx* +
동일 이름 *.module.css* 짝으로 이미 구성됨

페이지 & 레이아웃 — client/src

pages/ (Home · Input · Processing · Result · Adjust)

layouts/AppShell — 공통 화면 뼈대

styles/tokens.css — 디자인 토큰

react-router-dom 으로 페이지 라우팅 연결

5 개 페이지가 실제로 라우팅까지 연결된
동작하는 앱 (*react-router-dom*)

프로토타입 소개



홈 → 정보 입력 → 처리 중 → 결과 → 스케줄 조정 , 5 개 화면을 직접 눌러볼 수 있는 목업

배포된 프로토타입

<https://dodeho.github.io/hub/>

로컬 개발 서버

<http://localhost:5173/>

어떻게 계산할까 — 모델 개념



졸음은 쌓인다 (Process S)

깨어있는 시간이 길수록 졸음 (수면압) 이 쌓이고, 자는 동안 풀린다



생체시계가 있다 (Process C)

하루 주기로 오르내리는 원래 정해진 각성 리듬이 따로 있다



카페인을 일시적으로 억제한다 (UMP)

카페인 졸음 신호를 일시적으로 눌러줘서, 원하는 시각에 각성도를 끌어올릴 수 있다

결합하면 : 졸음 + 생체리듬 + 카페인 효과를 합쳐서, " 언제 자고 언제 카페인을 마셔야 시험 시작 시각에 각성도가 최고가 되는지 " 를 역산한다 (Borbély 1982 + Vital-Lopez 2024, UMP)

어떻게 최적 스케줄을 찾을까 — 계산 과정 3 단계



① 시간 그리드 만들기

오늘부터 마지막 시험까지 30 분 단위로 쪼개,
깨어있기 / 잠자기 × 카페인 섭취 / 미선택로
나눔



② 후보마다 시뮬레이션

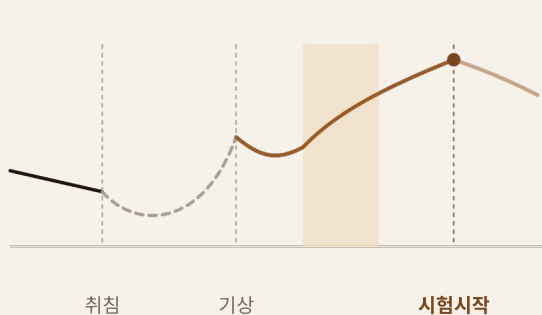
각 조합을 모델에 대입해 각성도 곡선을 계산,
시험 시각 각성도엔 가산점, 제약 위반엔 패널티



③ 최고점 스케줄 채택

전수탐색이 아니라 후보를 반복 개선하는 국소
탐색,
점수가 가장 높은 조합을 추천 스케줄로 제공

예시 : 이 과정을 거쳐 선택된 스케줄의 예측 각성도 곡선



■ 깨어있는 동안 예측 각성도

■ 수면 중 구간 (모델상 회복)

■ 카페인 부스트 구간

■ 정점 = 시험 시작 시각

다음 단계 · 고민

가장 고민되는 것



코드를 완전히 이해하고 있을까

비전공자라 도메인 지식이 없다 보니 ,
AI 가 만들어준 코드를 결과물에 쫓겨
그대로 쓰는 느낌 — 원리를 제대로
이해하고 가는 게 맞는지 고민



개인화까지 구현할 수 있을까

지금은 집단 평균 파라미터로 계산하는데 ,
개인 데이터가 쌓이면 더 정확한 결과를
낼 수 있을지 , 4 주 안에 어디까지
손댈 수 있을지 감이 안 옴



DB 를 어떻게 설계해야 할지 모르겠다

FE(React) · BE(Express) 는 정했는데 ,
Supabase 에 실제로 어떤 테이블 ·
스키마를 뒤야 할지 전혀 감이 안 옴